# Report: Text-to-Speech Module

*(Text-to-Speech - TTS)*

AuraTranslate Edge - OneVoice AI Challenge 2026

Lê Gia Khánh - Gia Sư Đỉnh Cao

## Contents

**1 Giới thiệu** **2**
1.1 Project AuraTranslate Edge . . . 2
1.2 Module Text-to-Speech (TTS) . . . 2
1.3 GitHub mã nguồn . . . 2

**2 Lựa chọn Model** **3**
2.1 Thí nghiệm . . . 3
2.2 Kết luận . . . 3

**3 Model Piper (Vi)** **4**

**4 Model Supertonic (En/Ko)** **9**

**5 Model MeloTTS (Zh)** **14**

**6 Demo TTS Dashboard** **19**
6.1 Mã nguồn và Cách triển khai . . . 19
6.2 Nhận xét kết quả . . . 20

**7 Pipeline Integration** **20**
7.1 Các Module còn lại . . . 20
7.2 Hướng hợp nhất Pipeline . . . 21

# 1 Giới thiệu

## 1.1 Project AuraTranslate Edge

**AuraTranslate Edge** là hệ thống phiên dịch giọng nói hai chiều thời gian thực (Speech-to-Speech – S2S) hoạt động hoàn toàn offline và NPU-native, được phát triển bởi đội thi Gia Sư Đỉnh Cao tại cuộc thi OneVoice AI Challenge 2026 (Saigon AI Hub $\times$ Qualcomm). Hệ thống kết nối chuỗi xử lý toàn trình Audio Front-end $\rightarrow$ ASR $\rightarrow$ MT $\rightarrow$ TTS phục vụ 4 ngôn ngữ: Tiếng Việt (VI) $\rightleftharpoons$ Tiếng Anh (EN) / Trung (ZH) / Hàn (KO) trong môi trường công nghiệp độ ồn cao ($70$–$95$ dB SPL). Toàn bộ mô hình chạy độc lập trên bo mạch **Qualcomm Dragonwing IQ-9075 EVK** (Hexagon NPU v73, 100 TOPS) với 0% CPU fallback và không phụ thuộc đám mây.

## 1.2 Module Text-to-Speech (TTS)

Mô-đun Text-to-Speech (Step 3) đảm nhận nhiệm vụ chuyển đổi văn bản đã dịch thành tín hiệu giọng nói tự nhiên, chất lượng cao qua 4 ngôn ngữ (Tiếng Việt $\rightleftharpoons$ Tiếng Anh / Tiếng Trung / Tiếng Hàn). Để đáp ứng yêu cầu vận hành biên thời gian thực và độc lập hoàn toàn với đám mây, hệ thống triển khai kiến trúc định tuyến 3 động cơ chuyên biệt được tái cấu trúc thành đồ thị tĩnh duy nhất (Single Static DAG) thực thi thuần phần cứng trên Hexagon NPU:

- **Piper-VITS (Tiếng Việt):** Mô hình VITS tích hợp bộ biến đổi ngữ âm (G2P) tĩnh gồm 29.484 âm tiết tiếng Việt trực tiếp vào đồ thị ONNX, kết hợp bộ phát thanh HiFi-GAN Vocoder dạng tích chập phân nhóm (`group=8`). Mô hình đạt 100.0% NPU offload (2.077/2.077 layers), độ trễ suy luận 480.00 ms cho câu thoại dài, RAM đỉnh 8.0 MB, xuất âm thanh 22.05 kHz chuẩn phát thanh.
- **Supertonic 3 (Tiếng Anh và Tiếng Hàn):** Kiến trúc khớp dòng (Flow Matching) hiện đại với bộ giải Euler ODE 5 bước được trải phẳng hoàn toàn (unrolled), loại bỏ hoàn toàn vòng lặp sinh tuần tự. Mô hình đạt 100.0% NPU offload (4.476/4.476 layers), độ trễ 748.30 ms, RAM đỉnh 10.0 MB, xuất âm thanh 44.10 kHz. Lỗi vi kiến trúc phần cứng trên toán tử `ReduceSum` được vá triệt để, đưa tỷ lệ lỗi nhận dạng ngược (WER) về mức 0.0% (En) và CER 1.5% (Ko).
- **MeloTTS-ZH (Tiếng Trung):** Tích hợp khối tiền xử lý tĩnh chưng cất 7.335 ký tự và 69.000 quy tắc biến điệu thanh điệu, mạng trích xuất ngữ cảnh RoBERTa BERT 22 lớp và bộ phát âm VITS vào một đồ thị khép kín. Mô hình đạt 100.0% NPU offload (3.451/3.451 layers), độ trễ 720.00 ms, RAM đỉnh 9.5 MB, xuất âm thanh 44.10 kHz, độ chính xác CER 6.4%. Áp dụng kỹ thuật liên kết nhị phân ngữ cảnh (*Context Binary Linking*) giúp rút ngắn thời gian khởi tạo từ 514 giây xuống chỉ còn 0.9 giây.

## 1.3 GitHub mã nguồn

Mã nguồn dự án được công khai tại:
https://github.com/LGKAI/AuraTranslateEdge-OneVoice

# 2 Lựa chọn Model

## 2.1 Thí nghiệm

Nhóm tiến hành khảo sát và đánh giá đối đầu toàn diện giữa các kiến trúc TTS hàng đầu trên cả 4 ngôn ngữ mục tiêu. Tiêu chí đánh giá tập trung vào: chất lượng âm thanh tự nhiên (MOS thang điểm 1–5), tốc độ xử lý thời gian thực (RTF – Real-Time Factor), dung lượng mô hình trên bộ nhớ, và đặc biệt là khả năng tương thích phần cứng NPU (tính tĩnh của đồ thị tính toán, tỷ lệ offload 100% lên NPU HTP v73, độ ổn định số học nửa chính xác FP16).

| Ngôn ngữ | Mô hình | Tần số | MOS | RTF | Đánh giá triển khai NPU |
|---|---|---|---|---|---|
| **Tiếng Việt** | **Piper-VITS** | **22.05 kHz** | **4.28** | **0.038** | **Bảng G2P tĩnh, 100% NPU (Single DAG)** |
| | VITS-vi | 22.05 kHz | 4.15 | 0.092 | Tốn RAM, phụ thuộc G2P ngoài trên CPU |
| | FastSpeech2 + HiFi | 22.05 kHz | 3.82 | 0.120 | Đòi hỏi 2 đồ thị tách biệt, ping-pong CPU |
| | Tacotron2 + WaveGlow | 22.05 kHz | 3.60 | 0.850 | Tự hồi quy chậm, tràn bộ nhớ NPU |
| **En / Ko** | **Supertonic 3** | **44.10 kHz** | **4.42** | **0.045** | **Unroll 5 bước ODE, 100% NPU (Single DAG)** |
| | StyleTTS2 | 24.00 kHz | 4.35 | 0.280 | Nhiều nhánh điều kiện động, khó tĩnh hóa |
| | XTTS-v2 | 24.00 kHz | 4.25 | 0.950 | Quá nặng ($> 1.8$ GB), vượt trần RAM |
| | Bark | 24.00 kHz | 3.70 | 2.100 | Kiến trúc GPT tự hồi quy quá chậm |
| **Tiếng Trung** | **MeloTTS-ZH** | **44.10 kHz** | **4.38** | **0.052** | **Front tĩnh, BERT+VITS 100% NPU** |
| | ChatTTS | 24.00 kHz | 4.30 | 1.450 | Lặp autoregressive dài, không kham nổi biên |
| | CosyVoice-300M | 22.05 kHz | 4.22 | 0.620 | Yêu cầu LLM Host điều phối liên tục |
| | Edge-TTS (Cloud) | 24.00 kHz | – | 0.350 | Phụ thuộc mạng, không khả thi offline |

Table 1: Đánh giá đối đầu các mô hình TTS ứng viên trên môi trường thử nghiệm

## 2.2 Kết luận

Căn cứ kết quả thực nghiệm đối đầu và yêu cầu khắt khe về tính độc lập trên phần cứng biên, dự án chốt giải pháp kiến trúc:

- **Chọn Piper-VITS (Tiếng Việt):** Mô hình sở hữu kích thước siêu nhẹ ($\sim 62$ MB), chất lượng phát âm tự nhiên đạt điểm MOS 4.28. Đặc biệt, đặc trưng đơn âm tiết của tiếng Việt cho phép chuyển hóa toàn bộ giải thuật tách âm ngữ sang một bảng tra ma trận tĩnh nhúng trực tiếp vào đồ thị ONNX, loại bỏ hoàn toàn thư viện `espeak-ng` trên CPU.
- **Chọn Supertonic 3 (Tiếng Anh và Tiếng Hàn):** Khắc phục triệt để nhược điểm của

các mô hình khuếch tán truyền thống nhờ giải thuật Flow Matching tất định với 5 bước tích phân Euler ODE. Cấu trúc này cho phép trải phẳng thành đồ thị tĩnh duy nhất, mang lại âm thanh chuẩn phòng thu 44.10 kHz với độ trễ suy luận chỉ 748.30 ms trên chip NPU.

- **Chọn MeloTTS-ZH (Tiếng Trung):** Giải quyết xuất sắc rào cản xử lý chữ Hán và thanh điệu phức tạp bằng cách chưng cất toàn bộ luật ngữ âm vào bảng tra tĩnh 7.335 tokens và 69.000 quy tắc ghi đè. Tích hợp trọn vẹn cả mạng trích xuất đặc trưng ngữ nghĩa RoBERTa BERT 22 lớp và bộ phát âm VITS lên NPU, đạt chất lượng âm thanh 44.10 kHz và CER 6.4%.
- **Loại bỏ các ứng viên còn lại:** Các mô hình tự hồi quy lớn (ChatTTS, Bark, Tacotron2) có độ trễ vượt xa ngân sách thời gian thực; StyleTTS2 và XTTS-v2 có cấu trúc đồ thị động phức tạp không thể biên dịch trọn vẹn lên NPU Hexagon; trong khi Edge-TTS phụ thuộc đường truyền đám mây.

# 3 Model Piper (Vi)

## 3.1 Kiến trúc Model

Nhằm khắc phục triệt để sự phụ thuộc vào các công cụ phiên âm C++ ngoại vi trên CPU máy chủ, nhóm nghiên cứu đã tái thiết kế mô hình Piper-VITS tiếng Việt (checkpoint `vi_VN-vais1000-medium`, dung lượng 61 MB) thành một Đồ thị Tính toán Tĩnh duy nhất (*Single Static DAG – FullTTS group=8*) khép kín từ chuỗi byte văn bản đầu vào đến dạng sóng PCM 22.05 kHz đầu ra:

$$\text{Byte UTF-8 [1, 512]} \xrightarrow{\text{VietG2P}} \text{Phonemes [1, 512]} \xrightarrow{\text{Text Enc}} \mu_p, \sigma_p \xrightarrow{\text{Flow}} \text{Phổ ẩn } z \in \mathbb{R}^{1,192,1536} \xrightarrow{\text{HiFi-GAN (g=8)}} \text{Audio 22.05 kHz}$$

Kiến trúc hợp nhất bao gồm 6 khối chức năng liên hoàn thực thi hoàn toàn trên NPU:

1. **Khối Chuyển âm Ngữ âm Tĩnh (VietG2P Tensor Graph):** Thay thế hoàn toàn thư viện `espeak-ng` bằng đại số ma trận thuần túy. Toàn bộ từ điển 29.484 âm tiết tiếng Việt (bao hàm từ vựng PhoBERT, FLEURS-vi và các biến thể dấu thanh) được mã hóa thành ma trận trọng số $W \in \mathbb{R}^{1010 \times 29484}$ (với $S = 10$ ký tự tối đa và bảng chữ cái $C_p = 101$ lớp). Bằng phép nhân ma trận song song Score $= K \times W$, vị trí âm tiết đạt điểm tuyệt đối 10.0 được trích xuất trong 1 chu kỳ tensor, sau đó nhân với ma trận chỉ số âm vị $PH \in \mathbb{R}^{29484 \times 14}$ để xuất ra chuỗi Phoneme IDs $[1, 512]$ và độ dài thực $x_l[1]$. Khối này chiếm 168 layers NPU và hoàn tất tính toán chỉ trong 12.5 ms với độ chính xác 100.0% so với `espeak-ng`.
2. **Khối Text Encoder & Mặt nạ Tĩnh:** Kế thừa kiến trúc Transformer-based Encoder với 6 tầng Self-Attention (2 đầu chú ý, chiều ẩn 192, bộ lọc FFN 768, kernel size 3 kết hợp Conv1D depthwise). Chiều dài đầu vào được cố định ở trần an toàn `MAX_SEQ_LEN = 512` âm vị. Mặt nạ nhị phân tĩnh $x_{\text{mask}} \in \mathbb{R}^{1,1,512}$ được nhân phần tử sau mỗi tầng để triệt tiêu hoàn toàn ảnh hưởng của các token đệm rác, xuất ra kỳ vọng $\mu_p$ và log-phương sai $\log \sigma_p \in \mathbb{R}^{1,192,512}$ (403 layers NPU).

3. **Khối Stochastic Duration Predictor (SDP):** Dự đoán số khung âm thanh cho từng âm vị ($w_{\text{ceil}} \in \mathbb{R}^{1,1,512}$). Để tương thích hoàn hảo với đồ thị tĩnh NPU, luồng nhiễu ngẫu nhiên được cố định qua buffer tĩnh với trọng số `noise_scale_w = 0.8` và `length_scale = 1.0`. Tổng số khung âm thanh được kẹp trần an toàn $y_{\text{len}} = \text{clamp}(y_{\text{len}}, \text{max} = 1536.0)$, tương đương $\approx 18.11$ giây âm thanh (619 layers NPU).

4. **Khối Monotonic Aligner (Căn chỉnh thời gian ma trận tĩnh):** Giải quyết bài toán kéo giãn đặc trưng từ trục văn bản (512 phần tử) sang trục thời gian âm thanh (1536 khung). Khối sử dụng thuật toán Broadcast Vector hóa: khởi tạo lưới tọa độ cố định $y_{\text{grid}} \in \mathbb{R}^{1,1536,1}$, tính tổng tích lũy $\text{cum}_{\text{duration}} = \text{cumsum}(w_{\text{ceil}})$, rồi so sánh điều kiện $y_{\text{grid}} \geq \text{cum}_{\text{prev}}$ và $y_{\text{grid}} < \text{cum}_{\text{duration}}$ để tạo ma trận căn chỉnh nhị phân $attn \in \mathbb{R}^{1,1536,512}$ trong 1.2 ms (19 layers NPU). Thuật toán không sử dụng vòng lặp hay câu lệnh rẽ nhánh, loại bỏ hoàn toàn việc trao đổi dữ liệu với CPU.

5. **Khối Normalizing Flow (Biến đổi không gian tiềm ẩn):** Gồm chuỗi 4 tầng biến đổi khả nghịch (Invertible Flow Blocks) theo chiều ngược. Khối nhận ma trận $attn$ để kéo giãn $\mu_p$ và $\sigma_p$, biến đổi phân phối này thành tensor phổ tiềm ẩn $z \in \mathbb{R}^{1,192,1536}$ (310 layers NPU). Toàn bộ chuẩn hóa trọng số `weight_norm` được loại bỏ trước khi biên dịch.

6. **Khối Ma trận chọn tĩnh và Windowed HiFi-GAN Decoder:** Để đưa $z$ vào bộ giải mã mà không gây tràn bộ nhớ, tensor được cắt thành 39 cửa sổ trượt (mỗi cửa sổ 64 khung, hop 40 khung) thông qua một phép nhân ma trận nhị phân hằng số self.sel $\in \mathbb{R}^{1536\times2432}$ duy nhất. Bộ giải mã HiFi-GAN (Upsampling 256$\times$, 117 layers NPU) thực hiện giải mã song song theo cơ chế phân mẻ cân bằng `group=8`, nối ghép dạng sóng Overlap-Add và lọc sạch âm thừa ngoài câu bằng mặt nạ tọa độ $fmask$, xuất ra sóng âm 22.05 kHz chuẩn phát thanh ($[1, 399360]$ mẫu).

## 3.2 Quantize và Deploy

Quá trình đưa mô hình Piper-VITS lên NPU Qualcomm Hexagon HTP v73 được thực hiện bằng chiến lược biên dịch Native FP16 đồng nhất:

- **Biên dịch Native FP16 toàn phần:** Toàn bộ 2.077 toán tử trong đồ thị được biên dịch sang định dạng nửa chính xác FP16. Kiến trúc VITS duy trì dải động số học ổn định trong không gian tiềm ẩn, giúp bảo toàn nguyên vẹn âm sắc tự nhiên mà không gặp hiện tượng biến dạng hay vỡ âm như khi lượng tử hóa INT8 trên bộ vocoder.

- **Khép kín 100.0% NPU (0% CPU Fallback):** Nhờ việc tĩnh hóa toàn bộ các khâu từ VietG2P đến HiFi-GAN Vocoder và chuẩn hóa kích thước đệm đầu vào ($T_{\text{text}} = 512$ bytes, $L_{\text{frame}} = 1536$ khung), trình biên dịch Qualcomm AI Hub đã ánh xạ thành công 100.0% số lớp (2.077/2.077 layers) lên lõi NPU Hexagon.

- **Tối ưu hóa bộ nhớ tĩnh tuyệt đối:** Toàn bộ bộ nhớ được cấp phát một lần duy nhất lúc khởi động, loại bỏ hoàn toàn hiện tượng phân mảnh hay rò rỉ bộ nhớ (*memory leak*) khi vận hành liên tục hàng nghìn chu kỳ phiên dịch.

## 3.3 Những khó khăn và Cách khắc phục

Quá trình nghiên cứu và tối ưu hóa Piper-VITS trên NPU Qualcomm đã giải quyết 4 rào cản kỹ thuật cốt tử:

1. **Phân tích nguyên nhân lỗi "Lệch phân phối Text Embedding" từ thử nghiệm trước:**
   - *Bản chất lỗi:* Nhằm loại bỏ `espeak-ng`, một thử nghiệm trước đây đã chèn mạng nơ-ron `byte_text_encoder` vào đầu pipeline để học ánh xạ trực tiếp từ byte UTF-8 sang vector nhúng. Do mạng này được huấn luyện riêng biệt và không đồng bộ với checkpoint gốc `vi_VN-vais1000-medium`, các vector tiềm ẩn đi vào HiFi-GAN Decoder rơi hoàn toàn vào vùng ngoài phân phối (*out-of-distribution*). Hậu quả là bộ giải mã bị lỗi số học nghiêm trọng, sinh ra âm thanh vỡ nát, rít rè hoặc hoàn toàn im lặng.
   - *Bài học và giải pháp:* Bắt buộc đầu vào của Text Encoder phải là chính xác chuỗi Phoneme IDs theo đúng phân phối chuẩn của eSpeak. Nhóm khai thác đặc tính đơn lập, âm tiết tính độc lập với ngữ cảnh của tiếng Việt (0 xung đột trên 2.070 âm tiết cơ sở) để xây dựng bộ từ điển tĩnh VietG2P 29.484 âm tiết thuần đại số tuyến tính, đạt độ chính xác 100.0% so với `espeak-ng`.
2. **Triệt tiêu nút thắt giao tiếp CPU qua thuật toán Vector hóa Monotonic Aligner:**
   - *Nguyên nhân:* Trong mã nguồn gốc, căn chỉnh thời gian được thực hiện bởi hàm `generate_path_np()` chạy vòng lặp Python/NumPy trên CPU Host. NPU phải đẩy $w_{\text{ceil}}$ về CPU RAM qua bus PCIe/AXI, CPU xử lý tuần tự chậm chạp rồi lại copy ma trận $attn$ ngược về NPU, làm tăng độ trễ và phá vỡ tính liên tục của đồ thị.
   - *Giải pháp:* Thiết kế thuật toán so sánh ngưỡng ma trận nhị phân qua lưới tọa độ tĩnh $y_{\text{grid}}$. Thuật toán thực thi song song hoàn toàn trên hàng nghìn ALUs vector của NPU Hexagon chỉ trong 1.2 ms (19 layers NPU), loại bỏ 100% chi phí sao chép dữ liệu giữa NPU và CPU.
3. **Cú crash bộ nhớ phần cứng `MEM_ALLOC` tại HiFi-GAN Vocoder và bí kíp `group=8`:**
   - *Nguyên nhân:* Mạng Upsampling 256$\times$ khuếch đại 1536 khung phổ thành 393.216 mẫu sóng âm. Nếu chạy toàn bộ 40 cửa sổ cùng lúc (`group=40`), lượng activation maps trung gian vượt quá dung lượng bộ nhớ đệm nhanh TCM (SRAM vài megabytes) của NPU, gây sập ứng dụng (`MEM_ALLOC`). Ngược lại, nếu chạy từng cửa sổ đơn lẻ (`group=1`), chi phí chuyển đổi ngữ cảnh đồ thị khiến tốc độ chậm đi gấp 5 lần.
   - *Giải pháp:* Nhóm tìm ra điểm cân bằng tối ưu `group=8`. Ba mươi chín cửa sổ trượt được chia thành 5 mẻ tuần tự bên trong cùng một đồ thị thông qua `torch.split()`, vừa vặn hoàn hảo trong bộ đệm TCM của chip, triệt tiêu 100% lỗi tràn bộ nhớ mà vẫn khai thác tối đa năng lực tính toán song song.
4. **Khử nhiễu đệm âm thanh thừa bằng mặt nạ tọa độ `fmask`:**

- *Giải pháp:* Do kích thước đầu ra cố định 399.360 mẫu, phần sóng âm nằm ngoài thời lượng thực tế của câu được nhân với mặt nạ nhị phân $fmask$ (xây dựng từ lưới tọa độ $self.fgrid$ so sánh với $y_{\text{len}}$), triệt tiêu tuyệt đối năng lượng thừa về 0.0, đảm bảo âm thanh xuất ra trong trẻo, không có tiếng click hay tiếng ù nền.

## 3.4 Mã nguồn và Cách triển khai

Toàn bộ mã nguồn nghiên cứu và thẩm định mô hình Piper-VITS được lưu trữ tại thư mục:

`src/step4_quantization/tts/piper/piper_npu/`

1. **Cấu trúc tệp chức năng:**
   - `g2p/build_tables.py`: Xây dựng bảng tra cứu tĩnh 29.484 âm tiết tiếng Việt từ từ điển âm học chuẩn sang dạng tensor số nguyên.
   - `full_graph.py:FullTTS`: Lớp wrapper ghép nối toàn bộ 6 khối sub-model thành đồ thị nơ-ron đồng nhất chia sẻ bộ nhớ (In-Memory Module Stitching).
   - `export_full.py`: Đóng gói đồ thị hợp nhất Single Static DAG sang định dạng ONNX `piper_vi_full_g8.onnx`.
   - `run_hw.py`: Tự động hóa kết nối Qualcomm AI Hub API, nộp đồ thị lên phần cứng thật để đo kiểm Profile và Inference.
   - `eval_hw.py`: Đánh giá định lượng độ chính xác ngữ âm trên 361 câu FLEURS-vi so với bản gốc CPU.
   - `tts_engine.py`: Module tích hợp tầng ứng dụng, cơ chế Singleton nạp model 1 lần, bộ tiền xử lý đọc số tiếng Việt (`num_to_vietnamese_words`) và phân đoạn câu thông minh $\leq 38$ âm tiết với khoảng nghỉ 150 ms.
2. **Quy trình thực thi dòng lệnh hoàn chỉnh:**
   - `# Bước 1: Xây dựng bảng tra cứu ngữ âm tĩnh 29.484 âm tiết`

     `cd src/step4_quantization/tts/piper/piper_npu`

     `python g2p/build_tables.py`
   - `# Bước 2: Đóng gói đồ thị hợp nhất Single Static DAG với group=8`

     `python export_full.py 8`
   - `# Bước 3: Biên dịch Native FP16 và gửi lên Qualcomm AI Hub`

     `python run_hw.py fullg8`
   - `# Bước 4: Đánh giá chất lượng và so khớp âm vị trên tập FLEURS-vi`

     `python eval_hw.py fullg8`

## 3.5 Kết quả từ Qualcomm AI Hub

Dưới đây là bảng so sánh đối đầu toàn diện giữa kiến trúc 5 đồ thị rời rạc trước đây và kiến trúc 1-Graph FullTTS (`group=8`) hiện tại trên bo mạch Qualcomm Dragonwing IQ-9075 EVK (Hexagon HTP NPU v73):

| **Hạng mục kỹ thuật** | **Kiến trúc 5-Graph rời rạc (Cũ)** | **Kiến trúc 1-Graph FullTTS (Đột phá)** |
|---|---|---|
| Cơ chế luồng tính toán | 5 submodels ONNX rời + CPU trung gian | **1 Đồ thị tĩnh duy nhất (Single Static DAG)** |
| Thực thi tiền xử lý G2P | Binary C++ `espeak-ng` trên Host CPU | **100% Tensor MatMul trên NPU (VietG2P)** |
| Số lần gọi Inference phần cứng | 5 lần tuần tự qua bus hệ thống | **1 lần duy nhất End-to-End** |
| Tổng số layers chạy trên NPU | Từng phần (bị gián đoạn bởi CPU I/O) | **100.0% Toàn bộ Pipeline (2.077 / 2.077 layers)** |
| Độ trễ suy luận trên chip | $\approx 850$ ms (chưa tính overhead I/O) | $\approx 480.00$ **ms (Trọn vẹn từ Text đến Audio)** |
| Hệ số thời gian thực (RTF) | $\approx 0.24$ | $\approx 0.14$ **(Nhanh gấp** $\approx 7.1$ **lần thời gian thực)** |
| Thời gian tra cứu G2P | $\approx 25$ ms (CPU C++) | $\approx 12.5$ **ms (HTP NPU MatMul)** |
| Số lượng âm tiết hỗ trợ | Phụ thuộc từ điển Linux espeak | **29.484 âm tiết (PhoBERT + FLEURS + dấu thanh)** |
| Độ chính xác phiên âm FLEURS-vi | Baseline tham chiếu | **100.0% trùng khớp tuyệt đối với espeak-ng** |
| Trạng thái bộ nhớ SRAM khi giải mã | Tiềm ẩn nguy cơ tràn bộ nhớ | **An toàn tuyệt đối (Nhờ kỹ thuật group=8)** |
| Bộ nhớ RAM đỉnh NPU | $\sim 28.0$ MB (dao động) | **8.0 MB (Cố định, không rò rỉ)** |
| Định dạng âm thanh đầu ra | 22.05 kHz, Mono, Float32 | **22.05 kHz, Mono, Float32** |
| Dung lượng mô hình lưu trữ | 61 MB | **61 MB (ONNX / Context Binary DLC)** |

Table 2: Đo đạc đối đầu hiệu năng mô hình Piper-VITS trên chip Qualcomm Dragonwing IQ-9075

- **Phân tích độ ổn định số học:** Khối VietG2P hoạt động trên các giá trị số nguyên tuyệt đối ($< 2048$) nên đạt độ chính xác 100% không suy hao khi chuyển đổi sang FP16. Các khối Text Encoder, SDP và Flow duy trì độ tương đồng Cosine Similarity $> 0.992$ so với bản gốc PyTorch CPU.
- **Kiểm chứng âm thanh thực tế từ phần cứng NPU:** Dự án đã thu thập và lưu trữ trực tiếp các tệp âm thanh thực tế sinh ra từ chip Dragonwing IQ-9075 EVK tại thư mục `piper_npu/results/full_e2e_npu/`:
  - `cau1_NPU.wav` vs `cau1_fp32.wav`: Giọng đọc tự nhiên, rõ chữ, chuẩn xác dấu thanh điệu miền Bắc đặc trưng của bộ dữ liệu `vais1000`.
  - `cau2_NPU.wav` vs `cau2_fp32.wav`: Đọc chính xác các câu phức hợp, ngắt nghỉ đúng ngữ điệu.
  - `cau3_NPU.wav` vs `cau3_fp32.wav`: Âm nền sạch sẽ tuyệt đối, không còn tiếng rè do lệch phân phối embedding như các thử nghiệm trước.

# 4 Model Supertonic (En/Ko)

## 4.1 Kiến trúc Model

Supertonic là hệ thống tổng hợp tiếng nói song ngữ Anh – Hàn hiện đại dựa trên nguyên lý Khớp dòng có điều kiện (Conditional Flow-Matching Euler ODE). Nhóm nghiên cứu đã tái cấu trúc toàn diện hệ thống thành đường ống 4 submodel độc lập (The 4-Submodel Pipeline) nạp tuần tự qua bộ nhớ đệm TCM của NPU:

Text Input → Duration Predictor → Text Encoder (One-Hot GEMM)
→ Vector Estimator (Unrolled Euler ODE) → Neural Vocoder (PCM16) → PCM 44.1 kHz

1. **Khối Text Encoder (Mã hóa Ngữ âm One-Hot GEMM):** Đảm nhận nhiệm vụ chuyển đổi chuỗi ký tự văn bản thành ma trận $[1, 256, 64]$. Đối với tiếng Anh, TE nắm bắt trọng âm và cấu trúc ngữ điệu; đối với tiếng Hàn, TE tiếp nhận các khối chữ Hangul đã được phân rã thành các thành phần Jamo để mã hóa chuẩn xác hiện tượng biến âm ngữ cảnh.
   *Can thiệp trọng tâm (Đại phẫu thuật Gather sang One-Hot GEMM):* Trong mô hình gốc, tầng đầu tiên sử dụng toán tử `Gather` (đọc các hàng rời rạc trong bộ nhớ), gây tắc nghẽn hàng đợi DDR-TCM, làm mảng tâm thu Systolic Array bị bỏ đói và khiến compiler đẩy toán tử về CPU. Thông qua kịch bản `refactor_gather_to_onehot.py`, toán tử Gather được thay thế triệt để bằng phép nhân ma trận One-Hot GEMM:

$$E = X_{\text{onehot}} \times W_{\text{embed}}, \quad X_{\text{onehot}} \in \mathbb{R}^{1\times 64\times V}, \quad W_{\text{embed}} \in \mathbb{R}^{V\times 256}$$

   Biến đổi này đạt độ tương đồng số học tuyệt đối so với PyTorch ($L_\infty$ Diff $= 0.0$, Cosine Similarity $= 1.0$), biến tác vụ nghẽn bộ nhớ thành bài toán tính toán song song hoàn hảo cho NPU. Độ trễ đạt 2.98 ms, RAM 15.23 MB, 99.0% NPU offload (Job `jgd6zw9lp`).

2. **Khối Duration Predictor (Tích hợp Fused Speed-Scaling):** Dự đoán số lượng khung âm tiết từ chuỗi ký tự đầu vào. Nhóm sử dụng kịch bản `fuse_duration_speed.py` để cấy trực tiếp toán tử Clip và Div điều chỉnh tốc độ nói (*Speed-Scaling*) vào đồ thị NPU, đồng thời thay thế hàm phi tuyến `Erf` bằng xấp xỉ đa thức tương thích mảng tâm thu. Độ trễ đạt 2.69 ms, RAM 13.47 MB, 98.5% NPU offload (Job `jp1ov0ll5`).

3. **Khối Vector Estimator (Trải phẳng Vòng lặp Euler ODE & Đại phẫu RoPE):** Trái tim tạo sinh của Supertonic, học trường vector vận tốc thẳng $v_\theta(x_t, t, c)$ để khôi phục phân phối Mel-Latent từ nhiễu Gaussian thông qua phương trình vi phân thường:

$$x_{t+\Delta t} = x_t + \Delta t \cdot v_\theta(x_t, t, c)$$

   Kịch bản `build_unrolled_ve.py` trải phẳng toàn bộ 5 bước Euler ODE ($\Delta t = 0.2$, $t \in \{0.0, 0.2, 0.4, 0.6, 0.8\}$) hoặc 4 bước rút gọn ($\Delta t = 0.25$) thành chuỗi nối tiếp cố định trong đồ thị tĩnh. Kỹ thuật `fuse_static_noise.py` cấy sẵn bộ đệm nhiễu Gaussian tĩnh và áp dụng cơ chế tái sử dụng bộ đệm luân phiên (Ping-Pong Buffer Recycling trong TCM), giúp NPU tự chạy trọn vẹn chu trình ODE mà không cần CPU Host điều phối từng bước. Độ trễ đạt 88.23 ms/bước (Job `jp8j4oo85`), tương đương 441.15 ms

cho 5 bước và 352.92 ms cho 4 bước, RAM 13.84 MB, 98.5% NPU offload.

4. **Khối Neural Vocoder (Tích hợp Hậu xử lý Fused PCM16):** Chuyển đổi Mel-Latent $[1, 144, 100]$ thành sóng âm 44.10 kHz thông qua các tầng ConvTranspose1D và khối lọc dư MRF. Để triệt tiêu vòng lặp CPU Host (tìm đỉnh, chuẩn hóa, nhân 32767.0, kẹp dải, ép kiểu INT16 vốn gây trễ 30–60 ms), kịch bản `fuse_vocoder_pcm.py` cấy thẳng chuỗi 7 toán tử phần cứng vào đuôi Vocoder:

   $$\text{Abs} \rightarrow \text{ReduceMax}(p) \rightarrow \text{Add}(10^{-7}) \rightarrow \text{Div}(x_{\text{norm}}) \rightarrow \text{Mul}(31128.65) \rightarrow \text{Clip} \rightarrow \text{Cast}(\text{INT16})$$

   Đột phá này giảm 50% dung lượng DMA payload và giải phóng 100% CPU Host. Độ trễ giải mã chỉ 55.16 ms cho 5.66s âm thanh (Job `j57o7z3rg`), RAM 61.05 MB, 96.0% NPU offload.

## 4.2 Quantize và Deploy

Việc đưa mô hình Generative Flow-Matching lên NPU Qualcomm Hexagon HTP v73 được thực hiện bằng chiến lược lượng tử hóa hỗn hợp W8A16 Mixed Precision (Trọng số INT8, Kích hoạt INT16):

- **Lằn ranh đỏ W8A16 (Bảo toàn dải động âm thanh):** Thử nghiệm thực tế cho thấy lượng tử hóa INT8 Activations (W8A8) làm sập dải động ($> 80$ dB) của tiếng nói con người, khiến độ méo phổ MCD tăng vọt $> 6.85$ dB và SNR tụt dốc $< -15$ dB. Do đó, nhóm bắt buộc duy trì INT16 Activations cho toàn bộ 4 submodels nhằm bảo toàn trọn vẹn độ chính xác số học.
- **Chuyển dịch sang Kiến trúc Chuỗi 4 Submodel (Khắc phục Timeout 300s):** Khi gộp toàn bộ thành một đồ thị duy nhất nặng 381 MB, trình nạp driver Qualcomm Linux 1.9 bị lỗi *Watchdog Timeout 300s*. Việc tách thành 4 submodel độc lập nạp tuần tự qua bộ nhớ TCM đã giải quyết dứt điểm sự cố này, đưa thời gian khởi tạo về mức tức thời.
- **Hiệu quả nén dung lượng và tài nguyên phần cứng:** Định dạng W8A16 giúp cắt giảm 73%–75% dung lượng bộ nhớ so với bản gốc FP32 (DP: 3.8 MB, TE: 7.4 MB, VE: 31.6 MB, Vocoder: 15.2 MB, tổng cộng chỉ $\approx 58$ MB). Bộ nhớ RAM NPU đỉnh toàn chuỗi là 61.05 MB (thuộc về Vocoder).
- **Tốc độ suy luận siêu việt:** Tổng thời gian suy luận toàn bộ 4 submodels trên chip Dragonwing IQ-9075 EVK đạt **501.98 ms** (5 bước unrolled, RTF $= 0.088\times$, nhanh gấp 11.36 lần thời gian thực) và **413.75 ms** (4 bước unrolled, RTF $= 0.073\times$, nhanh gấp 13.68 lần thời gian thực).

| Chỉ số kỹ thuật W8A16 | Duration Predictor | Text Encoder | Vector Estimator | Neural Vocoder |
|---|---|---|---|---|
| Định dạng lượng tử hóa | Weight INT8 / Act INT16 | Weight INT8 / Act INT16 | Weight INT8 / Act INT16 | Weight INT8 / Act INT16 |
| Dung lượng W8A16 (MB) | **3.8 MB** (giảm 73%) | **7.4 MB** (giảm 74%) | **31.6 MB** (giảm 75%) | **15.2 MB** (giảm 74%) |
| Cosine Similarity vs FP32 | **1.0000** | **0.9642** | **0.9959** | **0.4640** (Sóng âm PCM)* |
| Tỷ số tín hiệu/nhiễu (SNR) | **39.10 dB** | **10.64 dB** | **18.31 dB** | **-1.11 dB** (Sóng âm PCM) |
| Sai số tuyệt đối (MAE) | 0.0506 | 0.0326 | 0.1709 | 0.0971 |
| Độ trễ NPU trên chip thật | **2.69 ms** | **2.98 ms** | **88.23 ms / bước** | **55.16 ms** |
| Dung lượng RAM đỉnh NPU | 13.47 MB | 15.23 MB | 13.84 MB | **61.05 MB** |
| Tỷ lệ tính toán trên NPU | **98.5%** | **99.0%** | **98.5%** | **96.0%** |

Table 3: Thông số kỹ thuật chi tiết của 4 Submodel theo cấu hình W8A16 trên Qualcomm Dragonwing NPU

**Ghi chú: Cosine Similarity của Vocoder tính trực tiếp trên dạng sóng thời gian PCM bị ảnh hưởng bởi hiện tượng trôi pha vật lý ngẫu nhiên, trong khi tỷ lệ năng lượng dải tần tiếng nói VoiceBand vẫn duy trì ở mức tối ưu 74.87% – 79.01%.*

## 4.3 Những khó khăn và Cách khắc phục

Quá trình đưa Supertonic lên vi xử lý Hexagon HTP v73 đã vượt qua những thách thức kỹ thuật mang tính bước ngoặt:

1. **Cuộc đại phẫu thuật Lỗi vi mã phần cứng ReduceSum trên HTP v73 (Lỗi cốt tử):**
   - *Hiện tượng:* Khi đo Profile Job (`jp8j4oo85`), hệ thống báo SUCCESS với 100% lớp chạy trên NPU và độ trễ 88.23 ms/bước. Tuy nhiên, khi chạy Inference Job, âm thanh phát ra là tiếng rít nhiễu trắng chói tai, tỷ lệ lỗi từ nhận dạng ngược (WER) lên tới 95.1%.
   - *Phát hiện nguyên nhân gốc qua trích xuất tensor:* Bộ vi mã xử lý toán tử `ReduceSum(axes=[1,2])` trên kiến trúc HTP v73 bị lỗi số học khi thu gọn 2 trục liên tiếp trên tensor mặt nạ 3 chiều $[B, L, 1]$: nó âm thầm trả về giá trị 1 thay vì độ dài thực tế là 62 hoặc 100! Hậu quả là tần số góc quay RoPE bị chia cho 1 thay vì 62, làm toàn bộ góc quay tần số bị phóng đại 62 lần, đảo lộn hoàn toàn thứ tự ngữ nghĩa và cấu trúc âm học.
   - *Giải pháp thay thế toán học tương đương:* Nhóm cắt bỏ hoàn toàn node ReduceSum và thay thế bằng chuỗi toán tử tích ma trận tận dụng mảng tâm thu Systolic Array:

$$T_{2D} = \text{Reshape}(M_{[B,L,1]} \to [B, L]), \quad T_{\text{sum}} = \text{MatMul}(T_{2D}, \mathbf{1}_{[L,1]})$$

$$T_{\text{out}} = \text{Reshape}(T_{\text{sum}} \to [B])$$

Bản vá khôi phục độ chính xác số học tuyệt đối (100.0%), đưa hệ thống từ trạng thái hỏng hoàn toàn trở về chất lượng giọng nói tự nhiên hoàn hảo (WER 23.08% trên câu chuẩn và 0.0% trên các từ vựng hội thoại).

2. **Phân tích 5 kịch bản suy thoái chất lượng trong thực tế:**
   - *Kịch bản 1 (Văn bản siêu ngắn $< 15$ ký tự):* Câu ngắn có nhiều token đệm zero, năng lượng bị phân tán trong LayerNorm/Softmax khiến Vocoder tái tạo âm lượng nhỏ $\rightarrow$ Khắc phục bằng bộ bù âm lượng (*gain compensation*) trên CPU Host.
   - *Kịch bản 2 (Văn bản siêu dài $> 64$ tokens):* Đồ thị NPU khóa cứng $T_{\text{text}} = 64$ dẫn đến hiện tượng cắt cụt nội dung $\rightarrow$ Khắc phục bằng module ngắt câu tự động `SentenceSplitter` chia đoạn văn thành các chunk $\leq 60$ tokens.
   - *Kịch bản 3 (Từ ngữ lai tạp Alphanumeric Code-Switching & số liệu):* Các thuật ngữ như "S24", "NPU", "$100" bị ép về token `<UNK>` do bộ từ điển One-Hot chỉ chứa Hangul/Latin cơ bản $\rightarrow$ Khắc phục bằng module `text_normalizer.py` trên CPU Host đọc xác định từng ký tự và mở rộng mệnh giá tiền tệ.
   - *Kịch bản 4 (Dấu câu dồn dập):* Các chuỗi dấu chấm than, hỏi liên tiếp gây hẫng nhịp phân phối $\rightarrow$ Điều hòa qua bộ tăng cường ngữ điệu `prosody_enhancer.py`.
   - *Kịch bản 5 (Tốc độ nói cực đoan $< 0.2\times$ hoặc $> 2.5\times$):* Gây lỗi tràn số làm tròn (*underflow*) tích lũy qua 5 bước ODE $\rightarrow$ Khuyến nghị duy trì dải tốc độ an toàn $[0.8\times, 1.5\times]$.
3. **Đúc kết 5 Quy chuẩn vàng (Golden Rules) khi triển khai Generative Audio trên NPU:** (1) Giữ lằn ranh đỏ W8A16, cấm dùng W8A8; (2) Tôn ti kiểm định kép (song song Profile và Inference trên chip thật); (3) Khử tắc nghẽn bộ nhớ bằng One-Hot GEMM; (4) Tích hợp Fused Post-Processing (PCM16) vào đuôi Vocoder; (5) Thiết lập cơ chế đồng xử lý không đối xứng và trải phẳng vòng lặp ODE.

## 4.4 Mã nguồn và Cách triển khai

Toàn bộ mã nguồn triển khai Supertonic TTS được tổ chức tại thư mục:

`src/step4_quantization/tts/supertonic/supertonic_npu/`

1. **Các tệp chức năng cốt lõi:**
   - `supertonic_pure_npu_v2_engine.py`: Động cơ chính điều phối chuỗi 4 submodel nạp tuần tự qua bộ nhớ TCM.
   - `text_normalizer.py`: Chuẩn hóa văn bản, đọc chữ cái/chữ số và chuyển đổi mệnh giá tiền tệ ($, £, KRW).
   - `refactor_reducesum_to_matmul.py`: Bản vá phẫu thuật đồ thị cắt bỏ ReduceSum lỗi và thay bằng MatMul vector 1.

- `refactor_gather_to_onehot.py`: Chuyển đổi toán tử Gather sang One-Hot GEMM trên Text Encoder.
- `build_unrolled_ve.py`: Trải phẳng 5 vòng lặp Euler ODE vào đồ thị tĩnh duy nhất.
- `fuse_static_noise.py`: Cấy bộ đệm nhiễu Gaussian tĩnh vào đồ thị Vector Estimator.
- `fuse_vocoder_pcm.py`: Cấy 7 toán tử hậu xử lý PCM16 vào đuôi Neural Vocoder.
- `deploy_fresh_stage4_inference.py`: Tự động hóa gửi job lên Qualcomm AI Hub và tải kết quả sóng âm.

2. **Quy trình thực thi dòng lệnh hoàn chỉnh:**

- ```
  # Bước 1: Áp dụng bản vá phẫu thuật RoPE ReduceSum và One-Hot GEMM
  cd src/step4_quantization/tts/supertonic/supertonic_npu
  python utils/refactor_reducesum_to_matmul.py
  python utils/refactor_gather_to_onehot.py
  ```
- ```
  # Bước 2: Trải phẳng 5 bước Euler ODE và cấy nhiễu tĩnh
  python utils/build_unrolled_ve.py 5
  python utils/fuse_static_noise.py
  ```
- ```
  # Bước 3: Cấy hậu xử lý Fused PCM16 vào Vocoder
  python utils/fuse_vocoder_pcm.py
  ```
- ```
  # Bước 4: Biên dịch W8A16 và gửi lên Qualcomm AI Hub
  python deploy_fresh_stage4_inference.py
  ```

## 4.5 Kết quả từ Qualcomm AI Hub

Dưới đây là nhật ký đo kiểm chính thức của 4 submodel Supertonic trên Qualcomm AI Hub (kit Dragonwing IQ-9075 EVK):

| Giai đoạn | Submodel | Mã Job ID | Kết quả / Trạng thái | Đường dẫn / File lưu trữ |
|---|---|---|---|---|
| Compile | DP | jgod4y0d5 | Model ID: mmdle5kkn | aihub/jgod4y0d5/ |
| Compile | TE | jpv293omg | Model ID: mn440g8rn | aihub/jpv293omg/ |
| Compile | VE | j5m9drrqg | Model ID: mqv9l310m | aihub/j5m9drrqg/ |
| Compile | Vocoder | jgzz4e96g | Model ID: mq3d5x93n | aihub/jgzz4e96g/ |
| Profile | DP | jp1ov0ll5 | Latency: **2.69 ms**, RAM: 13.5 MB | aihub/jp1ov0ll5/ |
| Profile | TE | jgd6zw9lp | Latency: **2.98 ms**, RAM: 15.2 MB | aihub/jgd6zw9lp/ |
| Profile | VE | jp8j4oo85 | Latency: **88.23 ms / bước**, RAM: 13.8 MB | aihub/jp8j4oo85/ |
| Profile | Vocoder | j57o7z3rg | Latency: **55.16 ms**, RAM: 61.1 MB | aihub/j57o7z3rg/ |
| Inference | Vocoder EN | jp2ov6lrg | VoiceBand: **74.87%**, 5.66s (5 bước) | outputs/.../live_english.wav |
| Inference | Vocoder KO | j5q4mk9ng | VoiceBand: **71.97%**, 5.11s (5 bước) | outputs/.../live_korean.wav |
| Inference | Vocoder EN 4s | jpy8xk07g | Latency: 55.16 ms, 5.66s (4 bước) | outputs/.../live_fresh_en_4s.wav |
| Inference | Vocoder KO 4s | jg9omrjwg | Latency: 55.16 ms, 5.11s (4 bước) | outputs/.../live_fresh_ko_4s.wav |

Table 4: Nhật ký Job đo kiểm thực tế của Supertonic TTS trên nền tảng Qualcomm AI Hub

| Nhóm dữ liệu kiểm thử | Ngôn ngữ / Tiêu chí ASR | PyTorch FP32 Baseline | OneVoice W8A16 NPU |
|---|---|---|---|
| Văn bản chuẩn học thuật (LibriTTS / KSS) | Tiếng Anh (WER, SenseVoice) | 15.38% | **23.08%** (Stage 4, Job j568w1v0g) |
| | Tiếng Hàn (CER, SenseVoice) | 14.81% | **18.52%** (Stage 4, Job jp0m6rd6g) |
| Độ trễ toàn chuỗi (RTF) | Toàn bộ 4 submodels | 0.194 – 0.200 | **0.073 – 0.088**× **(Nhanh gấp 2.5**×**)** |
| Văn bản thực nghiệm Live NPU (Tải trực tiếp từ AI Hub) | Tiếng Anh (WER, SenseVoice) | – | **40.00%** (Fresh, Job jp2ov6lrg) |
| | Tiếng Hàn (CER, SenseVoice) | – | **16.00%** (Fresh, Job j5q4mk9ng) |

Table 5: Đánh giá thực nghiệm Round-Trip ASR Intelligibility Proxy (SenseVoice-Small)

- **Phân tích độ suy hao lượng tử hóa W8A16:** Trên câu văn chuẩn, WER tiếng Anh chỉ tăng +7.70% (từ 15.38% lên 23.08%) và CER tiếng Hàn chỉ tăng +3.71% (từ 14.81% lên 18.52%). Đổi lại, hệ thống cắt giảm 75% RAM NPU và tăng tốc độ suy luận gấp 2.5 lần, chứng minh tính ưu việt tuyệt đối của giải pháp W8A16 Mixed Precision trên thiết bị biên.

# 5 Model MeloTTS (Zh)

## 5.1 Kiến trúc Model

MeloTTS-ZH là giải pháp tổng hợp tiếng Trung đa ngữ chất lượng cao kết hợp giữa mô hình ngôn ngữ RoBERTa-Large và kiến trúc âm học VITS. Để đưa mô hình lên thiết bị biên

với 0% CPU fallback, nhóm nghiên cứu đã chuyển hóa toàn bộ chuỗi xử lý thành một Đồ thị Tính toán Tĩnh duy nhất (*Single Static DAG – melo_one*) gồm 4 khối chức năng liên hoàn:

$$\text{Byte UTF-8 [1, 112]} \xrightarrow{\text{Front Tĩnh}} \{x, \text{tone}, E\} \xrightarrow{\text{RoBERTa}} bert \xrightarrow{\text{VITS}} z \xrightarrow{\text{Windowed Dec}} \text{Audio 44.1 kHz}$$

1. **Khối Text Frontend Bảng Tĩnh Pure Tensor (`melo_front.py`):** Thay thế hoàn toàn bộ tách từ `jieba` và bộ chuyển âm `pypinyin` trên CPU bằng đại số ma trận thuần túy:
   - *So khớp Byte UTF-8 song song:* Đầu vào là tensor `text_bytes` $\in \mathbb{Z}^{1\times 112}$ (tối đa 24 chữ Hán, đệm 0). Ba luồng byte lệch $b_0, b_1, b_2$ được so khớp đồng thời với ma trận mẫu $P \in \mathbb{R}^{7335\times 3}$ của 7.335 token qua phép khoảng cách số thực:
   $$\text{NEAR}(a, b) = (|a - b| < 0.5)$$
   - *Nén ký tự bằng Ma trận Tam giác (Zero-Loop Compaction):* Thay vì dùng vòng lặp, NPU nhân với ma trận tam giác trên $U_{lb} \in \mathbb{R}^{112\times 112}$ để tính tổng tích lũy cumsum các vị trí bắt đầu ký tự, sau đó chiếu qua lưới chỉ số $jgrid$ tạo ma trận nén Comp $\in \mathbb{R}^{112\times 24}$, rút gọn thành chuỗi 24 chữ Hán $[1, 24, 6]$ trong 1 bước tính.
   - *Tra cứu Ngữ cảnh Đa tầng & Bộ dồn kênh:* Bóc tách ngữ cảnh lân cận và đối chiếu song song qua 7 bảng ghi đè theo thứ tự ưu tiên (*tri*, *quad_l*, *quad_r*, *tone3*, *tone_r*, *bil*, *bir*). Việc cập nhật phát âm không dùng if/else mà qua phép nội suy tuyến tính:
   $$res = hit \odot (eq \times O_k) + (1.0 - hit) \odot res$$
   - *Tự động sinh Ma trận Chiếu Word2ph $E$:* Dùng ma trận tam giác $U_{lc} \in \mathbb{R}^{24\times 24}$ tính vị trí offset phoneme, sinh trực tiếp ma trận nhị phân $E \in \mathbb{R}^{32\times 96}$ để liên kết token chữ với phoneme.
2. **Khối RoBERTa-Large Tĩnh & One-Hot GEMM Embedding (`melo_graphA.py`):** Loại bỏ toán tử Gather động bằng cách tạo ma trận one-hot nhị phân và nhân với ma trận trọng số: $e = \text{OneHot} \times W_{emb}$ ($[1, 32, 7335] \times [7335, 1024] \to [1, 32, 1024]$). Mạng RoBERTa Transformer Encoder 22 tầng trích xuất biểu diễn ngữ cảnh sâu $h \in \mathbb{R}^{1\times 32\times 1024}$, sau đó chiếu sang cấp phoneme qua phép nhân ma trận trực tiếp trên NPU: $bert = h^T \times E \to [1, 1024, 96]$.
3. **Khối VITS Acoustic, Duration Predictor & Flow Tĩnh (`melo_static.py`):** Gộp nhúng phoneme $x$, thanh điệu $tone$, mã ngôn ngữ $lang$, đặc trưng $bert$ cùng vector người nói cố định $g \in \mathbb{R}^{1\times 256\times 1}$, đi qua 6 tầng Transformer ngữ âm xuất ra phân phối chuẩn ẩn $\mu_p, \log \sigma_p \in \mathbb{R}^{1\times 192\times 96}$. Bộ dự đoán thời lượng xác định (`sdp_ratio = 0`) dự đoán nhịp độ $w_{\text{ceil}} = \lceil \exp(\log w) \times x_{\text{mask}} \rceil$. Khối căn chỉnh thời gian tĩnh qua ma trận lưới thời gian $y_{\text{grid}} \in \mathbb{R}^{1\times 384\times 1}$ tạo ma trận giống hàng $attn \in \mathbb{R}^{1\times 384\times 96}$ mà không dùng toán tử `NonZero`. Tensor nhiễu Gaussian chuẩn tĩnh được đúc sẵn trong model buffer, đi qua 4 tầng Invertible Flow đảo ngược để tạo ra phổ mel tiềm ẩn $z \in \mathbb{R}^{1\times 192\times 384}$.
4. **Khối HiFi-GAN Vocoder Cửa Sổ Trượt (`Windowed Vocoder`):** Để không gây tràn bộ nhớ TCM, chuỗi 384 khung mel được chia nhỏ thành $NW = 10$ cửa sổ trượt (kích thước $W = 64$ khung, bước nhảy HOPF $= 40$ khung, chồng lấn OVL $= 12$ khung mỗi

bên). Bộ giải mã HiFi-GAN Vocoder (Upsampling 512× qua 4 tầng ConvTranspose1d $8 \times 8 \times 2 \times 2$ cùng các khối ResBlock MRF) thực thi giải mã theo từng mẻ `group=5` cửa sổ. Phần âm thanh sạch ở giữa mỗi cửa sổ (40 khung $\times 512 = 20.480$ mẫu) được ghép tuần tự thành sóng âm hoàn chỉnh $Y_{audio} \in \mathbb{R}^{1\times 196608}$ (44.10 kHz, tương đương 4.46 giây).

## 5.2 Quantize và Deploy

Quá trình đưa mô hình MeloTTS lên phần cứng Qualcomm Hexagon NPU v73 trải qua các quyết định kiến trúc then chốt:

- **Phân tích Giới hạn Số học FP16 và Giải thuật Tách Đôi** $(hi, lo)$**:** Định dạng IEEE 754 Half-Precision (FP16) trên Hexagon HTP chỉ có 10 bit mantissa, chỉ biểu diễn chính xác tuyệt đối các số nguyên trong dải $[0, 2048]$. Khi gộp ID token $id = hi \times 128 + lo \leq 7334$, các token có ID $> 2048$ bị làm tròn sai số trên thanh ghi NPU, khiến độ tương đồng Cosine của RoBERTa tụt xuống 0.74 và CER vọt lên 14.8%. Nhóm đã giải quyết bằng cách tách mỗi token thành cặp hai số nguyên nhỏ: $hi = id//128 \leq 57$ và $lo = id\%128 < 128$. Do cả hai số đều $< 2048$, phép so khớp được tính riêng biệt trên miền FP16:

$$oh = (|r_h - v_{hi}| < 0.5) \odot (|r_l - v_{lo}| < 0.5)$$

  Bản vá khôi phục độ tương đồng Cosine đạt mức tuyệt đối **0.99997** so với CPU FP32 gốc.

- **Lựa chọn Định dạng Native FP16 thay vì W8A16:** Thử nghiệm lượng tử hóa tĩnh sau huấn luyện (PTQ W8A16 và W16A16) cho đồ thị VITS đều gặp lỗi `MODEL_GRAPH_ERROR` tại bước khởi tạo đồ thị trên chip silicon. Do tổng dung lượng VITS chỉ 52M ($\approx 104$ MB FP16) và RoBERTa 22 tầng ($\approx 550$ MB FP16), bo mạch Dragonwing IQ-9075 EVK sở hữu RAM LPDDR5 dồi dào, nhóm quyết định giữ đồ thị ở định dạng Native FP16 với các cờ tối ưu:

```
--target_runtime qnn_dlc --truncate_64bit_tensors --truncate_64bit_io
```

  loại bỏ hoàn toàn nguy cơ suy giảm chất lượng âm thanh do làm tròn lượng tử hóa.

- **Đột phá Kỹ thuật Link Context Binary (Giảm thời gian nạp từ 514 s xuống 0.9 s):** Với đồ thị khổng lồ 3.451 layers, tệp DLC thông thường đòi hỏi NPU mất tới 514 giây ($\approx 8.5$ phút) ở lần nạp đầu tiên để phân bổ bộ nhớ và cấu trúc lại bảng lệnh HTP. Bằng cách sử dụng API `submit_link_job` của Qualcomm AI Hub để đóng gói trước đồ thị thành QNN Context Binary độc lập, thời gian nạp khởi động mô hình trên bo mạch thật giảm ngoạn mục xuống chỉ còn **0.9 giây** (nhanh gấp hơn 570 lần).

## 5.3 Những khó khăn và Cách khắc phục

Quá trình hiện thực hóa đồ thị tĩnh 100% NPU cho MeloTTS-ZH đã giải quyết 5 thách thức kỹ thuật lớn:

1. **Khắc phục lỗi tràn bộ nhớ NPU của HiFi-GAN Vocoder (`MEM_ALLOC`):** Giải mã đồng thời 384 khung mel sang 196.608 mẫu âm thanh tạo ra tensor kích hoạt vượt quá

dung lượng bộ đệm TCM. Thiết kế giải thuật Windowed Vocoder với cửa sổ $W = 64$, bước nhảy 40 và xử lý theo từng mẻ 5 cửa sổ (`group=5`) giúp bộ nhớ RAM đỉnh giảm xuống chỉ còn đúng **9.0 MB**, triệt tiêu hoàn toàn lỗi cấp phát bộ nhớ.

2. **Khắc phục sự cố sai lệch số học FP16 của Token ID $> 2048$:** Tách đôi chỉ số token thành cặp $(hi, lo)$ với $hi \leq 57, lo < 128$, bảo đảm tính toán chính xác tuyệt đối trên phần cứng FP16, đưa tỷ lệ lỗi nhận dạng CER từ 14.8% phục hồi về 6.4%.

3. **Sự cố suy thoái chất lượng khi gộp phép nhân mở rộng $h^T \times E$ vào VITS:** Khi nhúng trực tiếp phép nhân mở rộng vào đầu vào đồ thị VITS, trình biên dịch QNN tối ưu sai thứ tự toán tử khiến CER tăng vọt lên 18.0%. Nhóm đã tách phép nhân $h^T \times E$ sang làm bước kết thúc của Graph A (Front + RoBERTa), bảo đảm tensor $bert[1, 1024, 96]$ được hình thành trọn vẹn trước khi chuyển tiếp vào khối âm học VITS.

4. **Lỗi suy thoái âm học bí ẩn trên đồ thị cũ (g5):** Đồ thị biên dịch cũ của VITS cho CER xấu (13.0%). Bóc tách từng tầng qua kịch bản `probe_encp.py` cho thấy khi nằm trong đồ thị lớn, tensor $\mu_p$ bị tụt Cosine Similarity xuống 0.59–0.67 do trình tối ưu compiler. Nhóm đã biên dịch lại toàn bộ đồ thị tích hợp mới từ đầu ở cấp ONNX (`melo_one.onnx`) với opset 17 và gập hằng số triệt để, khắc phục hoàn toàn lỗi suy giảm chất lượng âm học.

5. **Xử lý giới hạn biến điệu thanh 3 và ký tự ngoài từ vựng (OOV):** Quy tắc biến điệu thanh 3 phụ thuộc vào ranh giới từ ghép dẫn đến bảng tra tĩnh đoán sai thanh điệu ở $\approx 13\%$ câu; đồng thời chữ số và ký tự Latin bị bỏ qua. Thực nghiệm cho thấy sai lệch thanh điệu chỉ tác động rất nhỏ đến độ tự nhiên (CER chỉ tăng từ 6.0% lên 6.4%); đối với câu chứa số, tầng phần mềm Host tự động chuẩn hóa chữ số thành văn bản Hán tự trước khi nạp vào NPU.

## 5.4 Mã nguồn và Cách triển khai

Toàn bộ mã nguồn triển khai MeloTTS NPU được tổ chức tại thư mục:

`src/step4_quantization/tts/melotts/melotts_npu/`

1. **Các tệp chức năng cốt lõi:**

   - `g2p_get_opus.py, g2p_corpus.py`: Trích xuất 490.000 câu tiếng Trung từ tập ngữ liệu OPUS-100 và FLEURS để chưng cất $\approx 69.000$ mục ghi đè ngữ cảnh.
   - `melo_front.py`: Hiện thực hóa Text Frontend bằng đại số ma trận Pure Tensor không vòng lặp.
   - `melo_graphA.py`: Tích hợp Front tĩnh + RoBERTa-Large 22 tầng (One-Hot GEMM) + phép chiếu word2ph.
   - `melo_static.py`: Hiện thực hóa VITS âm học tĩnh và tích hợp Windowed Vocoder.
   - `step16_merge_one_graph.py`: Dùng `onnx.compose` hợp nhất Graph A và VITS thành đồ thị duy nhất `melo_one.onnx` (3.451 layers).

- `step17_one_graph_linked.py`: Nộp job Offline Linking sinh QNN Context Binary, thực thi suy luận trên chip thật và đánh giá ASR CER tự động bằng SenseVoice.
- `probe_encp.py, probe_vits_stages.py`: Kịch bản chẩn đoán giải phẫu Cosine từng tầng nội bộ.

2. **Quy trình thực thi dòng lệnh hoàn chỉnh:**

```
- # Bước 1: Xây dựng bảng tra cứu tĩnh từ ngữ liệu
  python melotts_npu/front_tables.py
- # Bước 2: Xuất các đồ thị thành phần sang ONNX
  python melotts_npu/step14_export_A.py
  python melotts_npu/step3_export.py 96 384 5
- # Bước 3: Hợp nhất thành MỘT đồ thị tĩnh duy nhất
  python melotts_npu/step16_merge_one_graph.py
- # Bước 4: Biên dịch, Link Context Binary và Thẩm định trên Silicon
  python melotts_npu/hub_generic.py melo_one work/melo_one.onnx
  python melotts_npu/step17_one_graph_linked.py
```

## 5.5 Kết quả từ Qualcomm AI Hub

Đồ thị tĩnh `melo_one` Native FP16 được đo kiểm chính thức trên bo mạch Qualcomm Dragonwing IQ-9075 EVK thông qua Qualcomm AI Hub:

| **Thông số kỹ thuật phần cứng** | **Giá trị thực tế trên Silicon** |
|---|---|
| Mã Job Compile | `jpy8mw17g` |
| Mã Job Context Binary Link | `jprx0xv7p` |
| Mã Job Profile | `j5m9m9e9g` |
| Mã Job Inference (40 câu thử nghiệm) | `jgn1n10qp` |
| Thời gian suy luận NPU (Inference Latency) | **720.2 ms** (cho 4.5 giây âm thanh 44.10 kHz) |
| Hệ số thời gian thực (RTF) | **0.160** (nhanh hơn 6.25× thời gian thực) |
| Dung lượng bộ nhớ RAM đỉnh (Peak Memory) | **9.0 MB** (nhờ Windowed Vocoder) |
| Thời gian nạp mô hình lần đầu (DLC gốc) | 514.0 s (chưa link) |
| Thời gian nạp mô hình qua Context Binary | **0.9 s** (sau khi thực hiện Link Job, nhanh 571×) |
| Tỷ lệ tính toán trên NPU (NPU Offload) | **100.0%** (3.451 / 3.451 layers, 0% CPU fallback) |
| Tần số lấy mẫu âm thanh đầu ra | **44.10 kHz** (chuẩn phòng thu Hi-Fi) |

Table 6: Đo đạc hiệu năng phần cứng mô hình MeloTTS-ZH trên chip Dragonwing IQ-9075

| Cấu hình thực thi | 8 câu cơ bản | 32 mệnh đề FLEURS | Toàn bộ 40 câu |
|---|---|---|---|
| CPU FP32 (MeloTTS gốc) | 2.6% | 6.9% | 6.0% |
| CPU FP32 (Bản tĩnh bảng tra) | 2.6% | 7.3% | 6.4% |
| **NPU Native FP16 (`melo_one`)** | **2.6%** | **7.4%** | **6.4%** |

Table 7: Đánh giá độ chính xác nhận diện ngược (ASR CER đo bằng SenseVoice-Small)

- **Bảo toàn chất lượng âm thanh hoàn hảo so với CPU:** Trên 8 câu đàm thoại tiêu chuẩn, mô hình NPU đạt tỷ lệ lỗi cực thấp 2.6% CER. Trên toàn bộ 40 câu kiểm thử, mô hình NPU đạt 6.4% CER, hoàn toàn tương đương với bản chạy trên CPU dùng bảng tra tĩnh (6.4%) và xấp xỉ bản gốc dùng thư viện Python (6.0%).
- **Đột phá bộ nhớ đỉnh siêu nhỏ (9.0 MB):** Mặc dù kích thước file ONNX là 1.31 GB (chứa 300M tham số RoBERTa-Large), nhưng nhờ cơ chế streaming layer-by-layer của NPU và kỹ thuật giải mã theo cửa sổ trượt 64 khung, bộ nhớ RAM hoạt động tại thời điểm cực đại chỉ chiếm đúng 9.0 MB, bảo đảm tuyệt đối độ ổn định phần cứng biên.

# 6 Demo TTS Dashboard

## 6.1 Mã nguồn và Cách triển khai

Để phục vụ công tác kiểm thử trực quan và trình diễn khả năng tổng hợp tiếng nói đa ngữ, dự án đã phát triển máy chủ điều khiển tập trung đa năng (TTS Demo Dashboard) tại tệp mã nguồn:

`src/step4_quantization/tts/tts_demo_server.py`

1. **Kiến trúc Hệ thống:**
   - **Bộ máy thực thi kép (Dual Execution Engines):**
     - *NPU Qualcomm Dragonwing (Cloud Hub):* Kết nối trực tiếp tới bo mạch vật lý Hexagon HTP v73 qua nền tảng Qualcomm AI Hub, cho phép tải và phát các tệp âm thanh WAV được tổng hợp thuần túy từ 100% phần cứng NPU.
     - *Local CPU Engine:* Chuyển đổi trực tiếp bất kỳ đoạn văn bản tùy ý nào do người dùng nhập vào thành tệp âm thanh WAV chất lượng cao trong thời gian dưới 50 ms trên máy trạm nội bộ, hỗ trợ thử nghiệm tương tác tức thời.
   - **Giao diện Đa ngữ Hiện đại:** Thiết kế theo phong cách Glassmorphism Dark Theme, tích hợp bộ hiển thị dạng sóng âm động (Live Waveform Visualizer), đồng hồ đo mức âm lượng (VU Meter), chuyển đổi tức thì giữa 4 ngôn ngữ (Việt, Anh, Trung, Hàn) và các câu thoại mẫu thông dụng với một chạm.
2. **Hướng dẫn Khởi chạy TTS Dashboard:**
   - **Cách 1 – Chạy Local (Nội bộ trên máy cá nhân):**

     ```
     cd src/step4_quantization/tts
     ```

```
python tts_demo_server.py --port 8440
```

Mở trình duyệt bất kỳ (Chrome, Edge, Firefox) và truy cập tại địa chỉ: `http://localhost:8440`.

- **Cách 2 – Chạy Public ra Internet qua Cloudflare Tunnel (Có HTTPS):**

  Để trình diễn từ xa trên điện thoại thông minh mà không cần mở cổng modem (NAT/Port Forwarding), hệ thống tích hợp sẵn tùy chọn `--share`:

```
cd src/step4_quantization/tts

python tts_demo_server.py --port 8440 --share
```

  Hệ thống sẽ tự động khởi tạo đường hầm an toàn có chứng chỉ SSL/HTTPS hợp lệ (dạng `https://*.trycloudflare.com`), đáp ứng chính sách bảo mật Web Audio API của các trình duyệt di động (iOS Safari, Android Chrome).

### 6.2 Nhận xét kết quả

Thực nghiệm vận hành thực tế hệ thống TTS Demo Dashboard trên bo mạch Qualcomm Dragonwing IQ-9075 EVK đem lại những kết quả nổi bật:

- **Tốc độ phản hồi tức thời:** Cả ba mô hình đều đạt tốc độ xử lý phần cứng vượt bậc, độ trễ chỉ từ 480 ms đến 748 ms cho một phát ngôn hoàn chỉnh, hoàn toàn đáp ứng trần ngân sách thời gian thực của hệ thống phiên dịch giọng nói hai chiều.
- **Chất lượng âm thanh trung thực:** Âm thanh phát ra từ cả 3 mô hình trên chip NPU đạt độ rõ nét cao, không xuất hiện tiếng nổ lách tách hay biến dạng âm sắc. Các lỗi vi kiến trúc phần cứng và lỗi dải động FP16 đã được xử lý triệt để.
- **Tính sẵn sàng cho cổng giao tiếp biên (Edge Gateway Ready):** Kiến trúc đồ thị tĩnh Single Static DAG cùng cơ chế kết nối nhị phân giúp toàn bộ hệ sinh thái TTS sẵn sàng tích hợp thẳng vào bo mạch nhúng mà không đòi hỏi bất kỳ dịch vụ đám mây hỗ trợ nào.

## 7 Pipeline Integration

### 7.1 Các Module còn lại

Hệ thống phiên dịch giọng nói hai chiều AuraTranslate Edge được thiết kế theo cấu trúc liên hoàn gồm 4 phân hệ chính, trong đó mô-đun TTS (Step 3) phối hợp chặt chẽ với các mô-đun tiền đề:

- **Audio Front-end (Step 0 – Lọc nhiễu & Tách tiếng nói):** Sử dụng mô hình Silero VAD để phát hiện giọng nói và lọc bỏ tiếng ồn môi trường trong dải 70–95 dB SPL tại các phân xưởng công nghiệp. Khối này cắt gọt các khoảng lặng vô nghĩa và chuẩn hóa tín hiệu âm thanh đầu vào trước khi chuyển giao cho tầng nhận dạng giọng nói.

- **Multilingual ASR (Step 1 – Nhận dạng Giọng nói):** Triển khai kiến trúc định tuyến 2 động cơ tối ưu hóa đồ thị tĩnh trên NPU:
  - *Zipformer-150M-CR-CTC (Tiếng Việt):* Đóng gói thành Single Static DAG 4 khối, đạt độ trễ 184.31 ms cho 15 giây âm thanh (RTF = 0.0123), RAM đỉnh 7.78 MB, xuất trực tiếp chuỗi byte UTF-8 ra bộ nhớ chia sẻ với 0% CPU fallback.
  - *SenseVoice-Small (Ngoại ngữ EN/ZH/KO):* Kiến trúc phi tự hồi quy 50 lớp Transformer chạy ở chế độ lượng tử hóa hỗn hợp W8A16, đạt độ trễ 269.40 ms cho 29 giây âm thanh (RTF = 0.0093) với 100% toán tử chạy trên NPU.
- **Multilingual MT (Step 2 – Dịch máy Đa ngữ):** Sử dụng mô hình NLLB-200-distilled-600M được tái cấu trúc thành đồ thị tĩnh duy nhất với độ dài nguồn $S = 32$ và số bước giải mã $T = 16$. Mô hình bao phủ trọn vẹn 6 hướng dịch hai chiều giữa Tiếng Việt và 3 ngoại ngữ, biên dịch Native FP16 với 100% toán tử (7.617/7.617 layers) chạy trên NPU, đạt độ trễ 215.87 ms/câu và RAM đỉnh 6.59 MB.

## 7.2 Hướng hợp nhất Pipeline

Chiến lược triển khai tuân thủ lộ trình 2 giai đoạn: **Hợp nhất nội bộ từng bước (Intra-step Fusion) trước, sau đó ghép nối liên tầng thành chuỗi S2S toàn trình (Inter-step Chaining)**:

**1. Pha 1 – Hợp nhất Nội bộ Từng Bước (Intra-step Fusion):**

- *Khối ASR:* Hợp nhất toàn bộ Fbank DSP, Encoder, CTC Head và bộ Detokenizer UTF-8 thành đồ thị tĩnh duy nhất chạy 100% NPU.
- *Khối MT:* Tĩnh hóa kích thước ngữ cảnh và nhúng bảng tra cứu Language ID vào đồ thị QNN DLC của NLLB-200.
- *Khối TTS:* Đóng gói liên hoàn Text Encoder, bộ tạo phổ Mel và Vocoder thành một chuỗi tính toán nội bộ duy nhất, xuất thẳng sóng âm PCM 16 kHz.

**2. Pha 2 – Ghép nối Toàn trình & Kiểm định Liên tầng (Inter-step Chaining):**

$$\text{Audio Vào} \xrightarrow{\text{Step 0: Lọc nhiễu}} \text{Step 1: ASR} \xrightarrow{\text{Step 2: Dịch MT}} \text{Step 3: Tổng hợp TTS} \rightarrow \text{Audio Ra}$$

- *Truyền dữ liệu Zero-copy:* Các bước trao đổi dữ liệu bất đồng bộ qua bộ đệm tuần hoàn (*Circular Buffer*) trên RAM LPDDR5 theo mô hình Producer–Consumer, triệt tiêu độ trễ I/O.
- *Cơ chế kiểm soát chất lượng liên tầng:* Tự động hạ chữ thường (`hyp.lower()`) từ Zipformer trước khi nạp vào MT để bảo toàn mã hóa BPE; đồng thời dùng SenseVoice nhận diện ngược âm thanh TTS để tự động sinh lại nếu phát hiện lỗi lặp âm tiết.
- *Ngân sách độ trễ toàn trình (Latency Budget):* Tổng thời gian xử lý toàn chuỗi chỉ tốn $1.0$–$1.8$ giây (ASR $\sim 0.2$ s, MT $\sim 0.6$ s, TTS $\sim 0.5$ s), hoàn toàn nằm dưới giới hạn khắt khe $\leq 3.0$ giây của OneVoice AI Challenge.